

Machine Learning On Graphs HW2

Sravanth Chowdary Potluri
und5uv
CS6501 - 016

March 24, 2025

1.1 Effect of Depth on Expressiveness

In the given graphs, each node starts with a 1D feature vector $x = [1]$, and we apply a simplified GNN with **sum aggregation**, no non-linearity, and no learned transformations.

Each node updates its embedding by summing its own previous embedding and those of its neighbors. The two red nodes have **different 5-hop neighborhoods**, but their structures start differing at **3 hops**, where one red node receives information from node 8 via a different connection, while the other does not.

Since node embeddings remain the same until their neighborhoods diverge, it takes a minimum of **3 layers of message passing** for their embeddings to become distinguishable.

Final Answer: 3 layers.

1.2 Random Walk Matrix

1

The given graph consists of 4 nodes with edges:

$$\{(1, 2), (1, 3), (2, 3), (3, 4)\}.$$

The degree of each node is:

$$\deg(1) = 2, \quad \deg(2) = 2, \quad \deg(3) = 3, \quad \deg(4) = 1.$$

Since a random walk selects a neighbor uniformly, the transition matrix M is constructed as follows:

- Node 1 has neighbors $\{2, 3\}$, transitioning with probability $1/2$ each.
- Node 2 has neighbors $\{1, 3\}$, transitioning with probability $1/2$ each.
- Node 3 has neighbors $\{1, 2, 4\}$, transitioning with probability $1/3$ each.
- Node 4 has only one neighbor $\{3\}$, transitioning with probability 1.

Thus, the random walk transition matrix M is:

$$M = \begin{bmatrix} 0 & \frac{1}{2} & \frac{1}{2} & 0 \\ \frac{1}{2} & 0 & \frac{1}{2} & 0 \\ \frac{1}{3} & \frac{1}{3} & 0 & \frac{1}{3} \\ 0 & 0 & 1 & 0 \end{bmatrix}.$$

2

Finding the Eigenvector of M with Eigenvalue 1 and Normalizing it

We need to find a vector $\mathbf{r}$ that satisfies the equation:

$$M\mathbf{r} = \mathbf{r}$$

where M is the random-walk transition matrix:

$$M = \begin{bmatrix} 0 & \frac{1}{2} & \frac{1}{2} & 0 \\ \frac{1}{2} & 0 & \frac{1}{2} & 0 \\ \frac{1}{3} & \frac{1}{3} & 0 & \frac{1}{3} \\ 0 & 0 & 1 & 0 \end{bmatrix}.$$

The eigenvector corresponding to eigenvalue 1 represents the **stationary distribution**, meaning that if a probability distribution over nodes follows $\mathbf{r}$, it remains unchanged under the transition matrix M .

The eigenvector equation $M\mathbf{r} = \mathbf{r}$ expands as:

$$0 \cdot r_1 + \frac{1}{2}r_2 + \frac{1}{2}r_3 + 0 \cdot r_4 = r_1, \quad (1)$$

$$\frac{1}{2}r_1 + 0 \cdot r_2 + \frac{1}{2}r_3 + 0 \cdot r_4 = r_2, \quad (2)$$

$$\frac{1}{3}r_1 + \frac{1}{3}r_2 + 0 \cdot r_3 + \frac{1}{3}r_4 = r_3, \quad (3)$$

$$0 \cdot r_1 + 0 \cdot r_2 + 1 \cdot r_3 + 0 \cdot r_4 = r_4. \quad (4)$$

For a random walk on an undirected graph, the stationary distribution is proportional to the degree of each node:

$$r_i \propto \deg(i)$$

The degrees in the graph are:

$$\deg(1) = 2, \quad \deg(2) = 2, \quad \deg(3) = 3, \quad \deg(4) = 1.$$

Thus, we set:

$$r_1 = 2x, \quad r_2 = 2x, \quad r_3 = 3x, \quad r_4 = x.$$

Since $\mathbf{r}$ must sum to 1, since the probability distribution must sum to 1, we have:

$$2x + 2x + 3x + x = 8x = 1 \quad \Rightarrow \quad x = \frac{1}{8}.$$

So the stationary distribution (before normalization) is:

$$\mathbf{r}_{\ell_1} = \left(\frac{2}{8}, \frac{2}{8}, \frac{3}{8}, \frac{1}{8} \right) = (0.25, 0.25, 0.375, 0.125).$$

The question requires an ℓ_2 -normalized eigenvector. We compute the norm:

$$\begin{aligned} \|\mathbf{r}_{\ell_1}\|_2 &= \sqrt{0.25^2 + 0.25^2 + 0.375^2 + 0.125^2} \\ &= \sqrt{0.0625 + 0.0625 + 0.140625 + 0.015625} \\ &= \sqrt{0.28125} \approx 0.5303. \end{aligned}$$

Now we divide each component by 0.5303:

$$\begin{aligned} \mathbf{r} &= \left(\frac{0.25}{0.5303}, \frac{0.25}{0.5303}, \frac{0.375}{0.5303}, \frac{0.125}{0.5303} \right) \\ &\approx (0.47, 0.47, 0.71, 0.24). \end{aligned}$$

$$\boxed{[0.47, 0.47, 0.71, 0.24]}$$

This is the ℓ_2 -normalized eigenvector of M with eigenvalue 1, confirming that it is the stationary distribution.

1.3 Relation to Random Walk (i)

The mean aggregator update rule is given by:

$$h_i^{(l+1)} = \frac{1}{|N(i)|} \sum_{j \in N(i)} h_j^{(l)}.$$

Here, $N(i)$ denotes the set of neighbors of node i . This means that the updated embedding of node i is the average of the embeddings of its neighbors.

Writing this in matrix form for all nodes:

$$h^{(l+1)} = D^{-1} A h^{(l)},$$

where:

- A is the adjacency matrix, where $A_{i,j} = 1$ if (i, j) is an edge, otherwise 0. - D is the diagonal degree matrix, with $D_{i,i} = \deg(i)$.

Since a uniform random walk moves from node i to one of its neighbors with equal probability, the probability of transitioning from node i to node j is:

$$M_{i,j} = \begin{cases} \frac{1}{\deg(i)}, & \text{if } (i, j) \text{ is an edge,} \\ 0, & \text{otherwise.} \end{cases}$$

Thus, the transition matrix M for the random walk is:

$$M = D^{-1} A.$$

This confirms that applying the mean aggregator in a GNN corresponds exactly to multiplying by the random-walk transition matrix $D^{-1}A$.

1.4 Relation to Random Walk (ii)

When we add a skip connection and split the update half-and-half between keeping the old embedding and averaging the neighbors, the update rule becomes:

$$h_i^{(l+1)} = \frac{1}{2}h_i^{(l)} + \frac{1}{2}\left(\frac{1}{|N(i)|} \sum_{j \in N(i)} h_j^{(l)}\right).$$

In matrix form, for all nodes at once, this is:

$$h^{(l+1)} = \left(\frac{1}{2}I + \frac{1}{2}D^{-1}A\right) h^{(l)},$$

where A is the adjacency matrix, D is the diagonal degree matrix ($D_{i,i} = \deg(i)$), and I is the identity matrix. Therefore, the **new random-walk transition matrix** M corresponding to this skip connection is:

$$M = \frac{1}{2}I + \frac{1}{2}D^{-1}A.$$

1.5 Over-Smoothing Effect

The update rule for the node embeddings at each layer is given by:

$$h_i^{(l+1)} = \frac{1}{|N(i)|} \sum_{j \in N(i)} h_j^{(l)}.$$

In matrix form, for all nodes simultaneously, this can be written as:

$$h^{(l+1)} = D^{-1}A h^{(l)},$$

where A is the adjacency matrix and D is the diagonal degree matrix with $D_{i,i} = \deg(i)$. The matrix $D^{-1}A$ is known as the random-walk transition matrix, which describes the probability of moving from one node to another in a single step of a uniform random walk.

This update equation is equivalent to applying the transition matrix $D^{-1}A$ repeatedly to the initial embeddings. Thus, after l layers of message passing, the embedding update can be written as:

$$h^{(l)} = (D^{-1}A)^l h^{(0)}.$$

Since the graph is assumed to be connected, there exists a path between any two nodes, meaning that a random walker can eventually reach any node in the graph starting from any other node. This ensures that the Markov chain defined by $D^{-1}A$ is **irreducible**, which means that no subset of nodes is isolated in terms of probability flow; in other words, every node can be reached from any other node in a finite number of steps.

Additionally, because the graph is non-bipartite, it does not exhibit a strict two-set structure where random walkers must alternate between two disjoint groups of nodes at every step. In a bipartite graph, the transition probabilities would cycle between two

disjoint node sets, preventing convergence. The absence of this structure guarantees that the Markov chain is **aperiodic**, meaning that the probability of returning to any node does not follow a fixed periodic cycle and is instead spread over different time steps.

According to the **Markov Chain Convergence Theorem**, any Markov chain that is both irreducible and aperiodic has a unique *stationary distribution*. This means that as $l \rightarrow \infty$, the probability distribution of a random walker stabilizes to a fixed distribution that no longer depends on the initial node. In the context of a GNN, this implies that repeated applications of the aggregation function will cause all node embeddings to converge to a common limiting distribution, making them indistinguishable from one another.

Applying this result to the embedding propagation, we see that as $l \rightarrow \infty$, all node embeddings $h_i^{(l)}$ converge to the same stationary distribution, causing them to become indistinguishable. This phenomenon is **over-smoothing**, where deep GNN layers cause all node embeddings to collapse to a constant vector, reducing their usefulness for distinguishing between different nodes in downstream tasks.

1.6 Learning BFS with GNN

To design a Graph Neural Network (GNN) that accurately simulates the breadth-first search (BFS) algorithm, we need to define three key components: the message function, the aggregation function, and the update rule.

The BFS algorithm works by expanding the set of visited nodes at each step. Initially, only the source node is marked as visited (having an embedding of 1), while all other nodes are unvisited (embedding of 0). At each step, a node is marked as visited if it has at least one neighbor that was visited in the previous step. This means that our GNN should propagate the "visited" state from each node to its neighbors iteratively.

The **message function** is responsible for determining what information a node sends to its neighbors. Since we only care about whether a node has been visited, the simplest and most effective choice is:

$$M(h_u^{(t)}, h_v^{(t)}, e_{u,v}) = h_u^{(t)}$$

This means that every node u simply sends its own current embedding $h_u^{(t)}$ to its neighbors. If a node is visited ($h_u^{(t)} = 1$), it sends a message of 1; otherwise, it sends 0.

The **aggregation function** determines how a node processes the messages received from its neighbors. To correctly replicate BFS behavior, a node should become visited if at least one of its neighbors was visited in the previous step. This can be achieved using a **max aggregator**:

$$\text{AGGREGATE}(\{h_u^{(t)} : u \in N(v)\}) = \max_{u \in N(v)} h_u^{(t)}$$

Since all node features are binary (0 or 1), this function returns 1 if any neighbor was visited in the previous step and 0 otherwise.

The **update rule** must ensure that once a node is marked as visited, it remains visited in subsequent steps. This is achieved by taking the maximum of the node's previous embedding and the aggregated neighbor embeddings:

$$h_v^{(t+1)} = \max\left(h_v^{(t)}, \text{AGGREGATE}(\{h_u^{(t)} : u \in N(v)\})\right).$$

This guarantees that a node remains in state 1 after it is visited and that it can transition from 0 to 1 only when a neighbor has been visited.

The overall working of this GNN model follows the BFS traversal mechanism. Initially, only the source node has an embedding of 1, while all other nodes have 0. In the first step, all neighbors of the source node receive messages of 1, causing their embeddings to become 1. In the next step, the neighbors of these newly visited nodes receive messages of 1, and the process continues layer by layer. This mirrors the BFS expansion, where each layer of neighbors is explored in sequential steps.

This approach ensures that after t steps, all nodes at a distance t or less from the source node will have an embedding of 1, while all other nodes remain at 0. Since BFS naturally explores nodes level by level, this GNN structure correctly learns to perform BFS without requiring any additional supervision beyond the input features and connectivity structure of the graph.

2.1 Simple Matrix Factorization

In a simple matrix factorization model, we approximate the adjacency matrix A by $Z^\top Z$, where each column of Z is the embedding for one node. In the encoder-decoder perspective of node embeddings, each node i is assigned an embedding vector $\mathbf{z}_i$ by the encoder. To predict $A_{i,j}$, the **decoder** simply takes the dot product of the two node embeddings:

$$\hat{A}_{i,j} = \mathbf{z}_i^\top \mathbf{z}_j.$$

Hence, the decoder is precisely the inner (dot) product between the embedding vectors of the two nodes.

2.2 Alternate Matrix Factorization

If our **decoder** is defined as a bilinear form $\mathbf{z}_i^\top W \mathbf{z}_j$ for some matrix W , then each node pair (i, j) is reconstructed by

$$\hat{A}_{i,j} = \mathbf{z}_i^\top W \mathbf{z}_j.$$

Collecting all node embeddings into the matrix Z , we can write these reconstructions in matrix form as

$$\hat{A} = Z^\top W Z.$$

Hence, the **objective function** for the corresponding matrix factorization becomes

$$\min_Z \left\| A - Z^\top W Z \right\|^2.$$

2.3 Relation to Eigen-decomposition

Recall that a symmetric matrix A can be written in its eigen-decomposition form

$$A = Q \Lambda Q^\top,$$

where Q is an orthonormal matrix (i.e., $Q^\top Q = I$) and Λ is a diagonal matrix whose diagonal entries are the eigenvalues of A .

When we approximate A as $Z^\top W Z$ in the bilinear factorization setup, this becomes equivalent to learning an eigen-decomposition precisely if

- (1) W is diagonal, (2) Z is an orthonormal matrix.

Under these two conditions,

$$Z^\top W Z \iff Q \Lambda Q^\top.$$

2.4 Multi-hop Node Similarity

Define a new adjacency-like matrix M that encodes multi-hop connectivity. One approach is to set

$$M = A + A^2 + \dots + A^k,$$

so that $M_{i,j}$ reflects how many paths (or whether there is at least one path) between nodes i and j of length up to k . To keep it simple, we can use a binary version of M , i.e., $M_{i,j} = 1$ if there is at least one path of length $\leq k$ connecting i and j , and 0 otherwise.

If we continue to use the same encoder (node embedding lookup) and a dot-product decoder, then the corresponding matrix factorization problem we want to solve is

$$\min_Z \|M - Z^\top Z\|^2.$$

In other words, we replace the usual single-hop adjacency matrix A with the multi-hop adjacency M , and proceed to approximate M by the embedding dot products $Z^\top Z$.

3.1 Isomorphism Check

Yes, the two graphs are **isomorphic**.

We construct a one-to-one correspondence between the nodes of the two graphs as follows:

- 1 $\leftrightarrow$ A
- 2 $\leftrightarrow$ D
- 3 $\leftrightarrow$ H
- 4 $\leftrightarrow$ E
- 5 $\leftrightarrow$ B
- 6 $\leftrightarrow$ C
- 7 $\leftrightarrow$ G
- 8 $\leftrightarrow$ F

We now show that this mapping preserves all edge relationships:

- Node 1 is adjacent to 5, 6, 7 $\Rightarrow$ A is adjacent to B, C, G
- Node 2 is adjacent to 5, 6, 8 $\Rightarrow$ D is adjacent to B, C, F
- Node 3 is adjacent to 5, 7, 8 $\Rightarrow$ H is adjacent to B, G, F

- Node 4 is adjacent to 6, 7, 8 $\Rightarrow$ E is adjacent to C, G, F

Conclusion: The two graphs are isomorphic because the proposed mapping shows a one-to-one correspondence between their nodes while preserving adjacency relationships and the edges also match up.

3.2 Aggregation Choice

We construct two very simple graphs $\mathcal{G}_1$ and $\mathcal{G}_2$, each with one “target” node and some neighbors. Our goal is to show that **mean** and **max** aggregators produce the same new feature for the two target nodes, while **sum** produces different values.

Graph $\mathcal{G}_1$:

- Nodes: $\{v_1, a, b\}$.
- Edges: $\{(v_1, a), (v_1, b)\}$.
- Initial features:

$$h_{v_1}^{(0)} = 0, \quad h_a^{(0)} = 2, \quad h_b^{(0)} = 2.$$

Graph $\mathcal{G}_2$:

- Nodes: $\{v_2, x, y, z\}$.
- Edges: $\{(v_2, x), (v_2, y), (v_2, z)\}$.
- Initial features:

$$h_{v_2}^{(0)} = 0, \quad h_x^{(0)} = 2, \quad h_y^{(0)} = 2, \quad h_z^{(0)} = 2.$$

We assume that $h_{v_1}^{(0)} = h_{v_2}^{(0)} = 0$, so both target nodes share the same initial feature value.

Case 1: Mean Aggregator

$$\text{AGGREGATE}_{\text{mean}} = \frac{1}{|N(v)|} \sum_{u \in N(v)} h_u^{(0)}$$

For $\mathcal{G}_1$, the neighbors of v_1 are $\{a, b\}$. Thus,

$$h_{v_1}^{(1)} = \frac{h_a^{(0)} + h_b^{(0)}}{2} = \frac{2 + 2}{2} = 2.$$

For $\mathcal{G}_2$, the neighbors of v_2 are $\{x, y, z\}$. Thus,

$$h_{v_2}^{(1)} = \frac{h_x^{(0)} + h_y^{(0)} + h_z^{(0)}}{3} = \frac{2 + 2 + 2}{3} = 2.$$

Hence, with the mean aggregator, both $\mathcal{G}_1$ and $\mathcal{G}_2$ yield 2 for their target nodes (v_1 and v_2).

Case 2: Max Aggregator

$$\text{AGGREGATE}_{\max} = \max_{u \in N(v)} \{h_u^{(0)}\}$$

For $\mathcal{G}_1$, the neighbors of v_1 have features $\{2, 2\}$, so

$$h_{v_1}^{(1)} = \max(2, 2) = 2.$$

For $\mathcal{G}_2$, the neighbors of v_2 have features $\{2, 2, 2\}$, so

$$h_{v_2}^{(1)} = \max(2, 2, 2) = 2.$$

Again, both target nodes end up with the same new feature 2 when using the max aggregator.

Case 3: Sum Aggregator

$$\text{AGGREGATE}_{\text{sum}} = \sum_{u \in N(v)} h_u^{(0)}$$

For $\mathcal{G}_1$:

$$h_{v_1}^{(1)} = h_a^{(0)} + h_b^{(0)} = 2 + 2 = 4.$$

For $\mathcal{G}_2$:

$$h_{v_2}^{(1)} = h_x^{(0)} + h_y^{(0)} + h_z^{(0)} = 2 + 2 + 2 = 6.$$

Now the updated features 4 and 6 are clearly different. Hence, with the sum aggregator, we obtain $h_{v_1}^{(1)} \neq h_{v_2}^{(1)}$.

Conclusion: In these simple examples, both the mean and max aggregators produce the same updated feature (2) for the target nodes in $\mathcal{G}_1$ and $\mathcal{G}_2$. However, the sum aggregator produces different results (4 in $\mathcal{G}_1$ vs. 6 in $\mathcal{G}_2$), thus illustrating that **sum** can distinguish between the target nodes in the two graphs, while **mean** and **max** cannot.

References

[1] CS6501-016 Class Slides.

Acknowledgment

This Homework Makes use of Generative AI models such as GPT 4o, o1 and o3 to refine the answers and make them better. However all the answers are original and are not plagiarized from any source. I acknowledge that i have not given or received unauthorized assistance on this homework.